

MOBILE APPLICATION DEVELOPMENT

Complete Question Bank Solutions

Units 1–5 & 7 | Android • Flutter • Dart

Silver Oak College of Engineering & Technology | CE | Sem 6

UNIT 1 — ANDROID OS

Q1 & Q5. Android OS — Definition, Key Features and Architecture

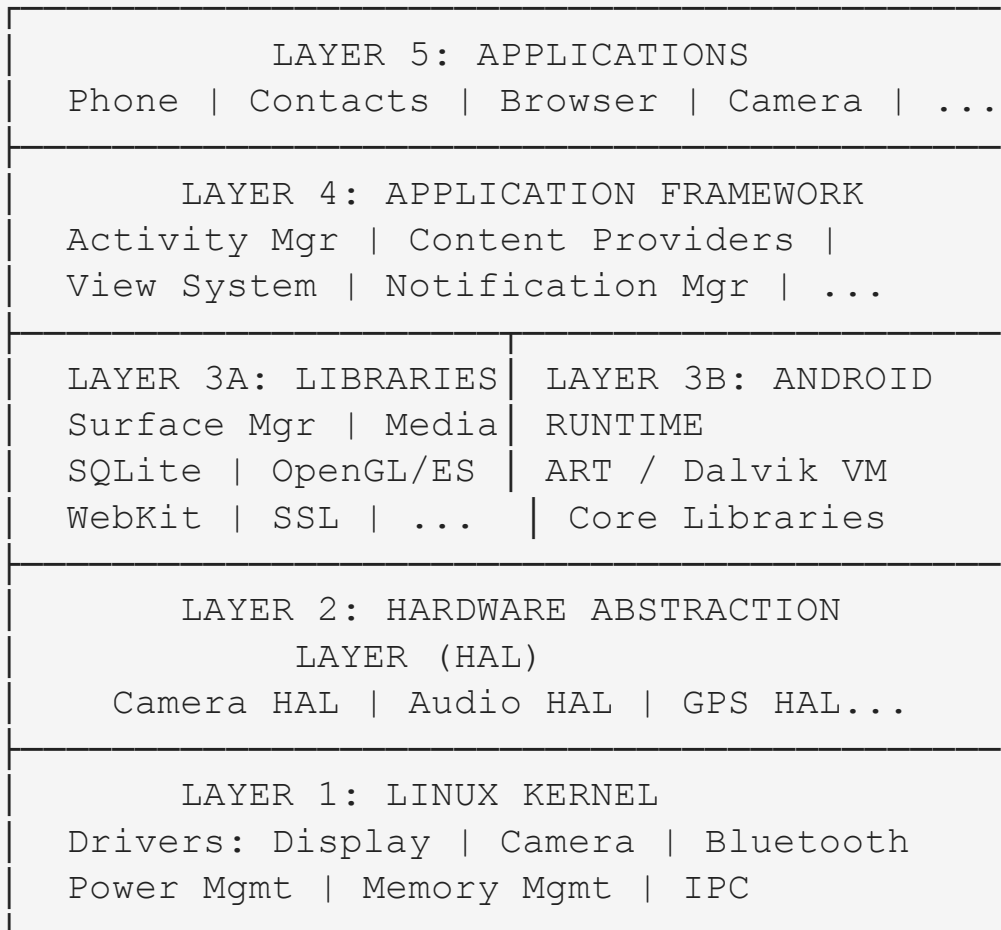
What is Android OS?

Android is an open-source, Linux-based mobile operating system developed by Google. It is designed primarily for touchscreen mobile devices such as smartphones and tablets. Android uses a modified Linux kernel and allows developers to write apps in Kotlin/Java.

Key Features of Android:

- Open Source — Based on Linux; source code freely available (AOSP)
- Multi-tasking — Multiple apps can run simultaneously
- Rich Application Framework — Reusable components for app development
- Google Play Store — Millions of apps available
- Connectivity — Wi-Fi, Bluetooth, 4G/5G, NFC support
- Media Support — MP3, MP4, JPEG, PNG and many formats
- SQLite Database — Built-in lightweight database
- Touch Interface — Supports gestures, multi-touch
- Push Notifications — Firebase Cloud Messaging (FCM)
- Hardware Access — Camera, GPS, accelerometer, sensors

Android Architecture (Layered Diagram):



Layer Descriptions:

Layer 1 — Linux Kernel:

- Foundation of Android; provides core system services
- Memory management, process management, device drivers
- Hardware abstraction between hardware and software

Layer 2 — Hardware Abstraction Layer (HAL):

- Standard interface between hardware drivers and Android framework

- Allows Android to work on different hardware without changing upper layers

Layer 3A — Native Libraries (C/C++):

- Surface Manager — Manages display and compositing
- Media Framework — Supports audio/video playback (MP3, MP4, AAC)
- SQLite — Lightweight relational database
- WebKit — Web browser engine
- OpenGL ES — 2D/3D graphics rendering
- SSL — Secure network communication

Layer 3B — Android Runtime (ART):

- ART (Android Runtime) replaced Dalvik VM from Android 5.0+
- Uses AOT (Ahead-Of-Time) compilation for faster app execution
- Core Java libraries included

Layer 4 — Application Framework:

- Activity Manager — Manages activity lifecycle and back stack
- Content Providers — Share data between applications
- View System — UI components (TextView, Button, RecyclerView)
- Notification Manager — Status bar notifications
- Location Manager — GPS and location services

Layer 5 — Applications:

- Top layer — built-in apps: Phone, Contacts, Browser, Camera
- Third-party apps installed from Play Store

Q2. Activity Lifecycle in Android

An Activity represents a single screen with a user interface. Android manages activities through a well-defined lifecycle of callback methods.

Lifecycle Methods:

onCreate():

- Called when activity is first created
- Initialize UI, set content view, bind data
- Called only once per activity creation

onStart():

- Activity becomes visible to the user
- Called after onCreate() or when activity returns from background

onResume():

- Activity is in the foreground and interactive
- Start animations, open camera, play audio

onPause():

- Another activity is coming to the foreground
- Release camera, pause animations, save draft data

onStop():

- Activity is no longer visible
- Release heavy resources (network connections, large objects)

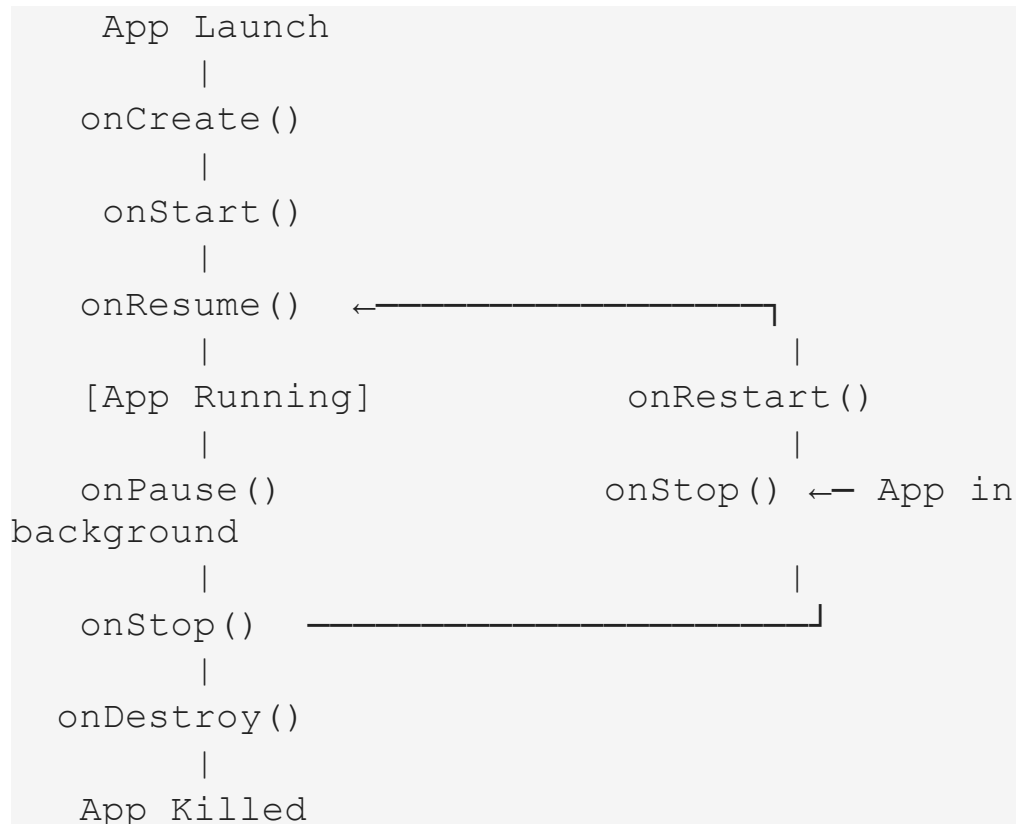
onRestart():

- Called when activity returns from stopped state

onDestroy():

- Activity is finishing or being destroyed by system
- Release all remaining resources

Lifecycle Diagram:



Code Example:

```
public class MainActivity extends
AppCompatActivity {
    @Override protected void onCreate(Bundle b) {
        super.onCreate(b);
        setContentView(R.layout.activity_main);
        Log.d("Lifecycle", "onCreate called");
    }
    @Override protected void onResume() {
        super.onResume();
        Log.d("Lifecycle", "onResume called");
    }
    @Override protected void onDestroy() {
        super.onDestroy();
        Log.d("Lifecycle", "onDestroy called");
    }
}
```

Q3. Fragment Lifecycle and Difference from Activity Lifecycle

A Fragment is a modular, reusable portion of a UI that lives inside an Activity. Fragments have their own lifecycle which is tied to the host Activity's lifecycle.

Fragment Lifecycle Methods:

- `onAttach()` — Fragment is attached to Activity; Activity reference available
- `onCreate()` — Fragment is created; initialize non-UI components
- `onCreateView()` — Inflate and return fragment's layout
- `onViewCreated()` — Called after view is created; bind views here
- `onStart()` — Fragment becomes visible
- `onResume()` — Fragment is interactive
- `onPause()` — Fragment going out of focus
- `onStop()` — Fragment not visible
- `onDestroyView()` — Fragment's view is being destroyed
- `onDestroy()` — Fragment itself is destroyed
- `onDetach()` — Fragment is detached from Activity

Fragment Lifecycle Diagram:

```
onAttach() → onCreate() → onCreateView() →  
onViewCreated()  
→ onStart() → onResume()  
→ onPause() → onStop() → onDestroyView()  
→ onDestroy() → detach()
```

Difference: Activity vs Fragment Lifecycle:

Feature	Activity Lifecycle	Fragment Lifecycle
Standalone	Yes — runs independently	No — needs a host Activity
Extra Methods	—	onAttach, onCreateView, onViewCreated, detach
UI Creation	setContentView() in onCreate()	Inflate layout in onCreateView()
Back Stack	Managed by OS	Managed by FragmentManager
Multiple per screen	One activity per screen	Multiple fragments in one activity
Lifecycle dependency	Independent	Tied to host Activity lifecycle

Q4. Intents in Android — Types and Examples

An Intent is a messaging object used to request an action from another component (Activity, Service, BroadcastReceiver). It is the primary way components communicate in Android.

Types of Intents:

1. Explicit Intent:

- Directly specifies the target component by class name
- Used to start a specific Activity or Service within the same app

```
// Start SecondActivity explicitly
```

```

Intent intent = new Intent(this,
SecondActivity.class);
intent.putExtra("name", "Raj");
startActivity(intent);

// In SecondActivity - retrieve data
String name = getIntent().getStringExtra("name");

```

2. Implicit Intent:

- Does not specify the target component; describes the action to perform
- Android resolves which app/component can handle the action
- If multiple apps can handle → shows chooser dialog

```

// Open a webpage
Intent intent = new Intent(Intent.ACTION_VIEW,
    Uri.parse("https://www.google.com"));
startActivity(intent);

// Make a phone call
Intent call = new Intent(Intent.ACTION_DIAL,
    Uri.parse("tel:9876543210"));
startActivity(call);

// Share text
Intent share = new Intent(Intent.ACTION_SEND);
share.setType("text/plain");
share.putExtra(Intent.EXTRA_TEXT, "Hello from my app!");
startActivity(Intent.createChooser(share, "Share via"));

```

Intent Filters:

- Declared in AndroidManifest.xml to let system know what actions a component can handle

```

<intent-filter>
    <action
        android:name="android.intent.action.VIEW"/>
    <category
        android:name="android.intent.category.DEFAULT"/>

```



```
<data android:scheme="https"/>
</intent-filter>
```

Feature	Explicit Intent	Implicit Intent
Target	Specific class name given	Action described; system resolves
Use case	Within same app	Across apps (share, dial, browse)
Example	<code>new Intent(this, B.class)</code>	<code>new Intent(ACTION_VIEW, uri)</code>

Q6. Steps to Create Hello Android Application

1. Install Android Studio from developer.android.com
2. Open Android Studio → New Project → Empty Activity
3. Configure Project:
 - Name: HelloAndroid
 - Package: com.example.helloandroid
 - Language: Java or Kotlin
 - Minimum SDK: API 21 (Android 5.0)
4. Project Structure Created:
 - MainActivity.java — Main Activity class
 - activity_main.xml — Layout file
 - AndroidManifest.xml — App configuration
 - build.gradle — Build configuration
5. activity_main.xml (Layout):

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout
xmlns:android="http://schemas.android.com/apk/res/
android"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <TextView
        android:id="@+id/tvHello"
```

```

        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello, Android!"
        android:textSize="24sp"
        android:layout_centerInParent="true"/>
</RelativeLayout>

```

6. MainActivity.java:

```

package com.example.helloandroid;
import androidx.appcompat.app.AppCompatActivity;
import android.os.Bundle;
public class MainActivity extends
AppCompatActivity {
    @Override
    protected void onCreate(Bundle
savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }
}

```

7. Run the App — Click green Run button or press Shift+F10
8. Choose emulator or connect physical device → App displays 'Hello, Android!'

Q7 & Q8. Android Platforms, Tools, Versions, SDK and Android Studio

Android Versions (Major):

Version	Name	API Level	Key Feature
Android 5.0	Lollipop	21	Material Design, ART runtime
Android 6.0	Marshmallow	23	Runtime Permissions
Android 7.0	Nougat	24	Multi-window, Doze mode
Android 8.0	Oreo	26	Picture-in-Picture, Autofill

Android 9.0	Pie	28	Adaptive Battery, Gesture Nav
Android 10	Q	29	Dark mode, Scoped Storage
Android 11	R	30	Chat Bubbles, Permissions auto-reset
Android 12	S	31	Material You, Privacy Dashboard
Android 13	Tiramisu	33	Per-app language, Predictive Back
Android 14	Upside Down Cake	34	Health Connect, Photo Picker

Android SDK (Software Development Kit):

- Collection of tools and libraries needed to develop Android apps
- Includes: Android API libraries, Android Emulator, ADB, DDMS, Build tools
- SDK Manager — Downloads specific API levels and tools
- Located at: ~/Android/Sdk on Linux/Mac, C:\Users\user\AppData\Local\Android\Sdk on Windows
- Key SDK tools: adb (Android Debug Bridge), emulator, sdkmanager, avdmanager

Android Studio:

- Official IDE for Android development — built on IntelliJ IDEA
- Features: Code editor with autocomplete, Layout Editor (drag & drop UI)
- Android Emulator — Virtual device for testing
- Logcat — Real-time log viewer for debugging
- Gradle Build System — Manages dependencies and builds APK
- Profiler — CPU, Memory, Network profiling tools
- AVD Manager — Create and manage virtual devices

- Instant Run / Apply Changes — Hot reload changes without full rebuild

Android Platforms:

- Android runs on phones, tablets, Android TV, Wear OS, Android Auto, Android Go
- Target Platform set in build.gradle using targetSdkVersion
- minSdkVersion — Minimum OS version app supports

UNIT 2 — ANDROID UI COMPONENTS, GRAPHICS AND ANIMATIONS

Q9. Creating Android UI and Types of Layouts

Android UI is built using XML layout files stored in `res/layout/`. The UI is a hierarchy of `View` and `ViewGroup` objects. Different layout managers arrange child views in different ways.

Types of Layouts:

1. `LinearLayout`:

- Arranges children in a single row (horizontal) or column (vertical)
- Attribute: `android:orientation` = 'horizontal' or 'vertical'
- Uses `android:weight` for proportional sizing

```
<LinearLayout android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <TextView android:text="Name"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"/>
    <EditText android:layout_width="match_parent"
        android:layout_height="wrap_content"/>
</LinearLayout>
```

2. `RelativeLayout`:

- Positions views relative to each other or the parent
- Attributes: `layout_alignParentTop`, `layout_toRightOf`, `layout_below`

```
<RelativeLayout ...>
    <TextView android:id="@+id/label"
        android:text="Name"
        android:layout_alignParentTop="true"/>
    <EditText android:layout_below="@id/label"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"/>
</RelativeLayout>
```

3. `ConstraintLayout`:

- Most flexible and efficient layout — recommended for complex UIs

- Views are positioned using constraints (connections) to other views or parent
- Reduces nested layouts → better performance

```
<androidx.constraintlayout.widget.ConstraintLayout
...>
    <Button android:id="@+id/btn"
        app:layout_constraintTop_toTopOf="parent"

        app:layout_constraintStart_toStartOf="parent"/>
</androidx.constraintlayout.widget.ConstraintLayout>
```

4. **FrameLayout:**

- Designed to hold a single child view, but can overlap multiple views
- Commonly used as a Fragment container

```
<FrameLayout android:id="@+id/fragment_container"
    android:layout_width="match_parent"
    android:layout_height="match_parent"/>
```

5. **GridLayout:**

- Arranges views in a grid of rows and columns
- Attributes: android:rowCount, android:columnCount

6. **TableLayout:**

- Organizes views into rows and columns like an HTML table
- Uses <TableRow> to define each row

Layout	Best For	Performance
LinearLayout	Simple row/column lists	Good
RelativeLayout	Relative positioning	Moderate
ConstraintLayout	Complex, flat UIs	Best
FrameLayout	Single view / Fragment host	Good
GridLayout	Grid arrangement	Good

Q10. **RecyclerView and CardView**

RecyclerView:

RecyclerView is an advanced, flexible, and efficient version of ListView for displaying large datasets. It recycles (reuses) views that scroll off screen, improving memory and performance.

Key Components of RecyclerView:

- RecyclerView — The container widget
- LayoutManager — Controls how items are arranged (LinearLayoutManager, GridLayoutManager, StaggeredGridLayoutManager)
- Adapter — Bridges data to views; contains ViewHolder
- ViewHolder — Holds references to item views to avoid repeated findViewById()

Advantages over ListView:

Feature	ListView	RecyclerView
ViewHolder	Optional	Mandatory (enforced)
Layout Managers	Only vertical list	Linear, Grid, Staggered
Animations	None built-in	ItemAnimator built-in
Performance	Poor for large data	Excellent (view recycling)
Item Click	setOnItemClickListener	Custom in Adapter

RecyclerView Example:

```
// In layout: activity_main.xml
<androidx.recyclerview.widget.RecyclerView
    android:id="@+id/recyclerView"
    android:layout_width="match_parent"
    android:layout_height="match_parent"/>

// Adapter
public class StudentAdapter extends
    RecyclerView.Adapter<StudentAdapter.ViewHolder> {
    List<String> list;
```

```

    StudentAdapter(List<String> list) { this.list =
list; }

    public static class ViewHolder extends
RecyclerView.ViewHolder {
        TextView tvName;
        ViewHolder(View v) {
            super(v);
            tvName = v.findViewById(R.id.tvName);
        }
    }

    @Override public ViewHolder
onCreateViewHolder(ViewGroup p, int t) {
        View v = LayoutInflater.from(p.getContext())
            .inflate(R.layout.item_student, p, false);
        return new ViewHolder(v);
    }

    @Override public void
onBindViewHolder(ViewHolder h, int i) {
        h.tvName.setText(list.get(i));
    }

    @Override public int getItemCount() { return
list.size(); }
}

// In Activity
RecyclerView rv = findViewById(R.id.recyclerView);
rv.setLayoutManager(new
LinearLayoutManager(this));
rv.setAdapter(new StudentAdapter(dataList));

```

CardView:

CardView is a widget from Material Design that wraps content in a card with elevation (shadow) and rounded corners. It is commonly used as the item layout inside RecyclerView.

- Provides elevation (shadow) and corner radius
- Part of CardView library (androidx.cardview)

```
<!-- item_student.xml -->
```



```

<androidx.cardview.widget.CardView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    app:cardElevation="4dp"
    app:cardCornerRadius="8dp"
    app:cardUseCompatPadding="true">
    <TextView android:id="@+id/tvName"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:padding="16dp"/>
</androidx.cardview.widget.CardView>

```

Q11. Custom Views in Android

A Custom View is a view created by extending an existing View class (View, ViewGroup, or existing widget) and drawing custom graphics or adding custom behavior.

When to Use Custom Views:

- When no existing view meets the UI requirement
- Reusable component needed across the app
- Complex drawing/animation required

Steps to Create a Custom View:

9. Create a class extending View
10. Override onDraw(Canvas canvas) to draw the custom UI
11. Override onMeasure() if custom sizing is needed
12. Add custom attributes in attrs.xml (optional)
13. Use it in XML layout or programmatically

Example — Custom Circle View:

```

public class CircleView extends View {
    private Paint paint;
    public CircleView(Context ctx, AttributeSet
attrs) {
        super(ctx, attrs);
        paint = new Paint();
        paint.setColor(Color.BLUE);
        paint.setAntiAlias(true);
    }
}

```

```

    }
    @Override
    protected void onDraw(Canvas canvas) {
        super.onDraw(canvas);
        int cx = getWidth() / 2;
        int cy = getHeight() / 2;
        int radius = Math.min(cx, cy) - 10;
        canvas.drawCircle(cx, cy, radius, paint);
    }
}

<!-- Usage in XML layout -->
<com.example.app.CircleView
    android:layout_width="200dp"
    android:layout_height="200dp"/>

```

Q12. UI Components and Event Handling in Android

Common UI Components (Views):

- TextView — Display non-editable text
- EditText — Input field for user text entry
- Button — Clickable button
- ImageView — Display images
- CheckBox — On/off toggle option
- RadioButton / RadioGroup — Single selection from group
- Switch — Toggle switch (on/off)
- SeekBar — Progress/slider control
- Spinner — Dropdown selection
- ProgressBar — Show loading progress
- ImageButton — Button with an image

Event Handling — 3 Ways:

Method 1: Anonymous Inner Class (most common):

```

Button btn = findViewById(R.id.btnSubmit);
btn.setOnClickListener(new View.OnClickListener()
{

```

```

@Override public void onClick(View v) {
    Toast.makeText(MainActivity.this,
        "Button Clicked!",
        Toast.LENGTH_SHORT).show();
}
});

```

Method 2: Lambda Expression (Java 8+):

```

btn.setOnClickListener(v -> {
    // handle click
    Log.d("TAG", "Button clicked");
});

```

Method 3: XML onClick attribute:

```

<Button android:onClick="onSubmitClick" ... />
// In Activity:
public void onSubmitClick(View view) {
    // handle click
}

```

Other Event Listeners:

- `setOnLongClickListener()` — Long press
- `setOnTouchListener()` — Touch events
- `addTextChangedListener()` — Text changes in EditText
- `setOnCheckedChangeListener()` — CheckBox/Switch changes

Q13. Android Menus and their Types

Menus provide additional options and actions to users. Android supports 3 types of menus:

1. Options Menu:

- Primary menu for an Activity — shown in the Toolbar/ActionBar
- Created in `res/menu/menu_main.xml` and inflated in `onCreateOptionsMenu()`
- Items handled in `onOptionsItemSelected()`

```

// menu_main.xml
<menu
xmlns:android="http://schemas.android.com/apk/res/
android">

```

```

        <item android:id="@+id/action_settings"
            android:title="Settings"
            android:showAsAction="ifRoom"/>
        <item android:id="@+id/action_about"
            android:title="About"/>
    </menu>
    // In Activity
    @Override public boolean onCreateOptionsMenu(Menu
    menu) {
        getMenuInflater().inflate(R.menu.menu_main,
        menu);
        return true;
    }
    @Override public boolean
    onOptionsItemSelected(MenuItem item) {
        if(item.getItemId() == R.id.action_settings) {

Toast.makeText(this, "Settings", Toast.LENGTH_SHORT)
.show();
        }
        return super.onOptionsItemSelected(item);
    }

```

2. Context Menu:

- Appears on long-press of a view — provides context-specific options
- Register view using `registerForContextMenu()`
- Created in `onCreateContextMenu()`, handled in `onContextItemSelected()`

```

registerForContextMenu(listView);
@Override public void
onCreateContextMenu(ContextMenu menu, View v,
    ContextMenu.ContextMenuInfo info) {
    super.onCreateContextMenu(menu, v, info);
    menu.add(0, 1, 0, "Edit");
    menu.add(0, 2, 0, "Delete");
}

```

3. Popup Menu:

- A modal menu anchored to a specific view — shown on button click

- Not tied to a list item; positioned near the anchor view

```
Button btnMenu = findViewById(R.id.btnMenu);
btnMenu.setOnClickListener(v -> {
    PopupMenu popup = new PopupMenu(this, btnMenu);

    popup.getMenuInflater().inflate(R.menu.popup_menu,
    popup.getMenu());
    popup.setOnMenuItemClickListener(item -> {
        Toast.makeText(this,
        item.getTitle(), Toast.LENGTH_SHORT).show();
        return true;
    });
    popup.show();
});
```

Menu Type	Trigger	Use Case
Options Menu	Toolbar / ActionBar	Global app actions (Settings, Search)
Context Menu	Long-press on view	Item-specific actions (Edit, Delete)
Popup Menu	Button click	Inline dropdown options

Q14. Media APIs in Android

Android provides MediaPlayer and MediaRecorder APIs for handling audio and video playback and recording.

Audio Playback using MediaPlayer:

```
MediaPlayer mp = MediaPlayer.create(this,
R.raw.music);
mp.start();    // Play
mp.pause();    // Pause
mp.stop();     // Stop
mp.release(); // Free resources – always call when
done
```

MediaPlayer States:

- Idle → Initialized → Prepared → Started → Paused → Stopped → Released

Audio from URL:

```
MediaPlayer mp = new MediaPlayer();
mp.setDataSource("https://example.com/audio.mp3");
mp.prepareAsync(); // Async prepare for network
sources
mp.setOnPreparedListener(m -> m.start());
```

Video Playback using VideoView:

```
<!-- In layout -->
<VideoView android:id="@+id/videoView"
    android:layout_width="match_parent"
    android:layout_height="300dp"/>

// In Activity
VideoView vv = findViewById(R.id.videoView);
vv.setVideoPath("/sdcard/video.mp4");
vv.setMediaController(new MediaController(this));
vv.start();
```

MediaRecorder — Record Audio:

```
MediaRecorder recorder = new MediaRecorder();
recorder.setAudioSource(MediaRecorder.AudioSource.MIC);
recorder.setOutputFormat(MediaRecorder.OutputFormat.THREE_GPP);
recorder.setAudioEncoder(MediaRecorder.AudioEncoder.AMR_NB);
recorder.setOutputFile("/sdcard/audio.3gp");
recorder.prepare();
recorder.start();
// ... later ...
recorder.stop(); recorder.release();
```

Q15. View Animations and their Types

Android provides rich animation support to improve user experience and make UIs feel alive.

Types of Animations:

1. Tween Animation (View Animation):

- Applies transformations to a View: translate, rotate, scale, alpha
- Defined in XML in res/anim/ folder

```
<!-- res/anim/slide_in.xml -->
<set
xmlns:android="http://schemas.android.com/apk/res/
android">
    <translate android:fromXDelta="-100%"
android:toXDelta="0%"
    android:duration="500"/>
</set>
// Apply in code:
Animation anim =
AnimationUtils.loadAnimation(this,
R.anim.slide_in);
myView.startAnimation(anim);
```

2. Frame Animation (Drawable Animation):

- Shows a sequence of Drawable images like a flipbook

```
<animation-list xmlns:android="..."
android:oneshot="false">
    <item android:drawable="@drawable/frame1"
android:duration="100"/>
    <item android:drawable="@drawable/frame2"
android:duration="100"/>
</animation-list>
```

3. Property Animator (ObjectAnimator / ValueAnimator):

- Animates any property of any object (not just Views)
- More powerful than tween animations

```
ObjectAnimator anim =
ObjectAnimator.ofFloat(view, "rotation", 0f, 360f);
anim.setDuration(1000);
anim.start();
```

4. Physics-Based Animations:

- SpringAnimation, FlingAnimation — simulate real-world physics

5. Transition Animations:

- Animate changes between layouts or activities

Types Summary:

Type	API	Used For
Tween/View Animation	AnimationUtils	Simple view transforms
Frame Animation	AnimationDrawable	Frame-by-frame sequences
Property Animator	ObjectAnimator	Any object property
ValueAnimator	ValueAnimator	Custom value transitions
Transition	TransitionManager	Layout/Activity transitions

Q16. Toolbar in Material Design

Toolbar is a versatile widget from Material Design that replaces the old ActionBar. It is more customizable and can be placed anywhere in the layout.

Advantages over ActionBar:

- Can be placed anywhere in the layout (not just top)
- Fully customizable — color, height, logo, navigation icon
- Can hold other views like search bars, spinners
- Works with CoordinatorLayout for scrolling effects

Setup:

- Use a NoActionBar theme in styles.xml
- Add Toolbar to layout, call `setSupportActionBar()` in Activity

Layout XML:

```
<androidx.appcompat.widget.Toolbar
    android:id="@+id/toolbar"
    android:layout_width="match_parent"
    android:layout_height="?attr/actionBarSize"
    android:background="?attr/colorPrimary"
    android:elevation="4dp"
```



```
android:title="My App"/>
```

Activity Code:

```
Toolbar toolbar = findViewById(R.id.toolbar);  
setSupportActionBar(toolbar);  
getSupportActionBar().setTitle("Home");  
getSupportActionBar().setDisplayHomeAsUpEnabled(true); // back button
```

styles.xml:

```
<style name="AppTheme"  
parent="Theme.AppCompat.Light.NoActionBar">  
    <item name="colorPrimary">#3F51B5</item>  
</style>
```

UNIT 3 — DATABASE CONNECTIVITY

Q17. Types of Storage in Android

Android provides multiple data storage options depending on the type, size, and privacy of data.

1. Internal Storage:

- Private storage for the app — other apps cannot access it
- Data deleted when app is uninstalled
- Use `Context.openFileOutput()` to write files

```
FileOutputStream fos = openFileOutput("data.txt",  
MODE_PRIVATE);  
fos.write("Hello".getBytes());  
fos.close();
```

2. External Storage:

- Shared storage — other apps can access (with permission)
- Use `Environment.getExternalStorageDirectory()` (deprecated in API 29+)
- Requires `WRITE_EXTERNAL_STORAGE` permission in Manifest
- In Android 10+: use `MediaStore` or `SAF` (Storage Access Framework)

3. Shared Preferences:

- Stores primitive data (String, int, boolean) as key-value pairs
- Lightweight; ideal for app settings and user preferences

```
SharedPreferences prefs =  
getSharedPreferences("MyPrefs", MODE_PRIVATE);  
prefs.edit().putString("username", "Raj").apply();
```

4. SQLite Database:

- Built-in relational database for structured data storage
- Use `SQLiteOpenHelper` to manage database creation and version management

5. Room Persistence Library:

- Abstraction layer over SQLite — cleaner, type-safe, annotation-based
- Google's recommended way to use databases

Storage Type	Data Type	Access	Best For
Internal Storage	Any file	App only	Private files
External Storage	Any file	Shared	Media, shared files
Shared Preferences	Key-Value primitives	App only	Settings
SQLite/Room	Structured tables	App only	Complex data

Q18. Shared Preferences — Storing and Retrieving Data

SharedPreferences is an interface that provides access to a persistent file of key-value pairs. It is ideal for saving small amounts of primitive data like user settings, login status.

Getting SharedPreferences:

```
// App-wide preferences:
SharedPreferences prefs =
    getSharedPreferences("MyPrefsFile",
MODE_PRIVATE);
// Activity-specific:
SharedPreferences prefs =
    getPreferences(MODE_PRIVATE);
```

Writing Data:

```
SharedPreferences.Editor editor = prefs.edit();
editor.putString("username", "Raj");
editor.putInt("age", 21);
editor.putBoolean("isLoggedIn", true);
editor.apply(); // async – preferred over commit()
```

Reading Data:

```
String name = prefs.getString("username",
"Guest"); // default=Guest
int age = prefs.getInt("age", 0);
boolean loggedIn = prefs.getBoolean("isLoggedIn",
false);
```

Removing Data:

```
editor.remove("username").apply();
editor.clear().apply(); // remove all
```

Full Example — Login Remember Me:

```
// Save on login
SharedPreferences.Editor ed =
    getSharedPreferences("login",
MODE_PRIVATE).edit();
ed.putString("user",
etUsername.getText().toString());
ed.putBoolean("remembered", true);
ed.apply();

// Check on app start
SharedPreferences sp =
getSharedPreferences("login", MODE_PRIVATE);
if(sp.getBoolean("remembered", false)) {
    String user = sp.getString("user", "");
    // auto-login user
}
```

Q19. Room Persistence Library — Architecture and Components

Room is Google's recommended database library for Android. It provides an abstraction layer over SQLite with compile-time verification of SQL queries.

Room Architecture:

```
[ Your App Code ]
    |
[ DAO (Data Access Object) ]
    |
[ Room Database ]
    |
[ SQLite Database ]
```

Three Main Components:

1. Entity — @Entity:

- Represents a table in the database

- Each field becomes a column

```
@Entity(tableName = "student")
public class Student {
    @PrimaryKey(autoGenerate = true)
    public int id;
    @ColumnInfo(name = "student_name")
    public String name;
    public int marks;
}
```

2. DAO — @Dao:

- Interface containing database operations as annotated methods
- Room generates the implementation automatically

```
@Dao
public interface StudentDao {
    @Insert void insert(Student s);
    @Update void update(Student s);
    @Delete void delete(Student s);
    @Query("SELECT * FROM student")
    List<Student> getAllStudents();
    @Query("SELECT * FROM student WHERE id=:sid")
    Student getById(int sid);
}
```

3. Database — @Database:

- Abstract class that extends RoomDatabase
- Contains entities list and database version

```
@Database(entities = {Student.class}, version = 1)
public abstract class AppDatabase extends
RoomDatabase {
    public abstract StudentDao studentDao();
    // Singleton instance
    private static AppDatabase instance;
    public static AppDatabase getInstance(Context
ctx) {
        if(instance == null) {
            instance = Room.databaseBuilder(ctx,
AppDatabase.class, "student_db")
```

```

        .allowMainThreadQueries() // avoid in
production
        .build();
    }
    return instance;
}
}

```

Usage in Activity:

```

AppDatabase db = AppDatabase.getInstance(this);
StudentDao dao = db.studentDao();
dao.insert(new Student(0, "Priya", 92));
List<Student> list = dao.getAllStudents();

```

Q20. Internal Storage vs External Storage

Feature	Internal Storage	External Storage
Privacy	Private to app	Shared / public
Permission needed	None	READ/WRITE_EXTERNAL_STORAGE
Available always	Yes	No (card may be removed)
On uninstall	Data deleted	Data may remain
Access by other apps	No	Yes (with permission)
Size	Limited (device storage)	Usually larger
API	openFileOutput(), getFilesDir()	getExternalFilesDir(), MediaStore
Use case	Private app data	Media files, shared documents

Q21 & Q23. Scoped Storage in Android — Benefits and Limitations

Scoped Storage was introduced in Android 10 (API 29) to limit broad access to shared external storage, improving user privacy and security.

What is Scoped Storage?

- Each app gets access only to its own app-specific directory on external storage
- Apps can access their own files freely but need permissions or MediaStore to access other files
- Reduces the ability of malicious apps to read/modify other apps' files

Benefits:

- Privacy — Apps cannot access all files on the device without user consent
- Security — Limits malicious apps from reading sensitive files
- No Permission needed — For app's own directory (getExternalFilesDir())
- Cleaner storage — Files auto-deleted when app is uninstalled
- MediaStore API — Controlled access to shared media (Photos, Music, Videos)

Limitations:

- Cannot freely browse all files — limited to specific directories
- File manager apps need `MANAGE_EXTERNAL_STORAGE` permission (special)
- Legacy apps (targeting API <29) may break if they rely on broad file access
- More complex code needed for accessing shared media
- Requires SAF (Storage Access Framework) for user-chosen files

Accessing Files with Scoped Storage:

```
// App-specific external dir - no permission
needed
File dir =
getExternalFilesDir(Environment.DIRECTORY_PICTURES
);

// Access media files via MediaStore
ContentResolver cr = getContentResolver();
Uri collection =
MediaStore.Images.Media.EXTERNAL_CONTENT_URI;
```

Q22. Runtime Permissions in Android

Starting from Android 6.0 (Marshmallow, API 23), dangerous permissions must be requested at runtime (while the app is running), not just declared in the Manifest.

Permission Categories:

- Normal Permissions — Granted automatically (INTERNET, VIBRATE, ACCESS_NETWORK_STATE)
- Dangerous Permissions — Must be requested at runtime (CAMERA, LOCATION, CONTACTS, READ_EXTERNAL_STORAGE)

Steps to Request Runtime Permission:

14. Declare permission in AndroidManifest.xml
15. Check if permission is already granted using `checkSelfPermission()`
16. If not granted, request using `requestPermissions()`
17. Handle result in `onRequestPermissionsResult()`

Example — Camera Permission:

```
// Step 1: Manifest
<uses-permission
android:name="android.permission.CAMERA"/>

// Step 2-3: Request in Activity
if(ContextCompat.checkSelfPermission(this,
Manifest.permission.CAMERA)
    != PackageManager.PERMISSION_GRANTED) {
    ActivityCompat.requestPermissions(this,
        new String[]{Manifest.permission.CAMERA},
        100);
} else {
    openCamera(); // permission already granted
}

// Step 4: Handle result
@Override public void
onRequestPermissionsResult(int requestCode,
```



```
        String[] permissions, int[] grantResults) {
    if(requestCode==100 && grantResults.length>0
        &&
grantResults[0]==PackageManager.PERMISSION_GRANTED
    ) {
        openCamera();
    } else {
        Toast.makeText(this,"Permission
Denied",Toast.LENGTH_SHORT).show();
    }
}
```

UNIT 4 — APPLICABILITY TO INDUSTRIAL PROJECTS

Q24 & Q27. Web Services in Android and JSON Parsing

Android apps frequently fetch data from web servers using REST APIs. The data is typically returned in JSON format and parsed into Java objects.

Steps to Fetch and Parse JSON:

18. Add INTERNET permission in Manifest
19. Make HTTP request (using HttpURLConnection or Retrofit)
20. Receive JSON response as String
21. Parse JSON using JSONObject or JSONArray
22. Display data in UI

AndroidManifest.xml:

```
<uses-permission  
    android:name="android.permission.INTERNET"/>
```

JSON Parsing Example:

```
// JSON Response from API:  
//  
{ "name": "Raj", "age": 21, "subjects": ["Math", "Science"  
    "]}  
  
try {  
    JSONObject obj = new JSONObject(jsonString);  
    String name = obj.getString("name");  
    int age = obj.getInt("age");  
    JSONArray subjects =  
obj.getJSONArray("subjects");  
    for(int i=0; i<subjects.length(); i++) {  
        Log.d("Subject", subjects.getString(i));  
    }  
} catch(JSONException e) { e.printStackTrace(); }
```

Fetching from URL using HttpURLConnection (AsyncTask pattern):

```
new Thread(() -> {  
    try {
```

```

        URL url = new
URL("https://api.example.com/students");
        HttpURLConnection conn = (HttpURLConnection)
url.openConnection();
        conn.setRequestMethod("GET");
        BufferedReader br = new BufferedReader(
            new
InputStreamReader(conn.getInputStream()));
        StringBuilder sb = new StringBuilder();
        String line;
        while((line=br.readLine())!=null)
sb.append(line);
        String json = sb.toString();
        // Parse json on UI thread
        runOnUiThread(() -> parseJson(json));
    } catch(Exception e) { e.printStackTrace(); }
}).start();

```

Q25. Retrofit Library in Android

Retrofit is a type-safe HTTP client library for Android developed by Square. It simplifies API communication by converting HTTP API into Java interfaces.

Advantages of Retrofit:

- Simple annotations — @GET, @POST, @PUT, @DELETE define API calls
- Automatic JSON parsing — with Gson/Moshi converter
- Asynchronous calls — built-in async with Callback
- RxJava / Coroutines support
- Better error handling than manual HttpURLConnection

Setup — build.gradle:

```

implementation
'com.squareup.retrofit2:retrofit:2.9.0'
implementation
'com.squareup.retrofit2:converter-gson:2.9.0'

```

Step 1 — Model Class:

```
public class Student {  
    public int id;  
    public String name;  
    public int marks;  
}
```

Step 2 — API Interface:

```
public interface ApiService {  
    @GET("students")  
    Call<List<Student>> getStudents();  
  
    @POST("students")  
    Call<Student> addStudent(@Body Student student);  
}
```

Step 3 — Retrofit Instance:

```
Retrofit retrofit = new Retrofit.Builder()  
    .baseUrl("https://api.example.com/")  
  
    .addConverterFactory(GsonConverterFactory.create()  
    )  
    .build();  
ApiService service =  
retrofit.create(ApiService.class);
```

Step 4 — Make API Call:

```
service.getStudents().enqueue(new  
Callback<List<Student>>() {  
    @Override public void  
onResponse(Call<List<Student>> call,  
            Response<List<Student>> response) {  
        if(response.isSuccessful()) {  
            List<Student> list = response.body();  
            // Update UI with list  
        }  
    }  
})  
@Override public void  
onFailure(Call<List<Student>> c, Throwable t) {  
    Log.e("Retrofit", "Error: " + t.getMessage());  
}
```

```
});
```

Q26 & Q28. MVVM Architecture and Comparison with MVC

MVVM (Model-View-ViewModel) is an architectural pattern promoted by Google for Android development. It cleanly separates UI from business logic.

MVVM Components:

Model:

- Represents data and business logic
- Fetches data from database (Room) or network (Retrofit)
- Has no reference to View or ViewModel

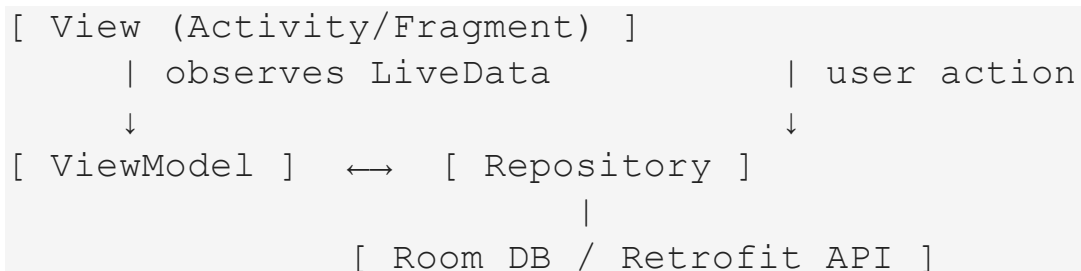
View:

- Activity, Fragment, XML layout — the UI layer
- Observes LiveData from ViewModel
- Never directly accesses the Model

ViewModel:

- Holds and manages UI-related data
- Survives configuration changes (screen rotation)
- Exposes data via LiveData or StateFlow
- Communicates with Repository (Model layer)

MVVM Flow Diagram:



Code Example:

```
// ViewModel
public class StudentViewModel extends
AndroidViewModel {
    private MutableLiveData<List<Student>> students;
    public StudentViewModel(Application app) {
super(app); }
}
```

```

    public LiveData<List<Student>> getStudents() {
        if(students==null) { students = new
MutableLiveData<>();
            loadStudents(); }
        return students;
    }
    private void loadStudents() {
        // fetch from Room or Retrofit and set value

students.setValue(AppDatabase.getInstance(getAppli
cation()))
        .studentDao().getAllStudents());
    }
}

// In Activity – observe data
StudentViewModel vm = new ViewModelProvider(this)
    .get(StudentViewModel.class);
vm.getStudents().observe(this, list -> {
    adapter.setData(list); // update RecyclerView
});

```

MVC vs MVVM Comparison:

Feature	MVC	MVVM
Full Form	Model-View-Controller	Model-View-ViewMode l
Controller/VM role	Controller handles input	ViewModel exposes data via LiveData
View-Model coupling	Tightly coupled	Loosely coupled
Data binding	Manual update of UI	Automatic via LiveData observers
Testability	Difficult (controller tied to UI)	Easy (ViewModel is pure Java)
Configuration change	Re-fetch data	ViewModel survives rotation

Complexity	Simpler	More structured
Android support	Not officially promoted	Google recommended

Q29. Publishing an Android App on Play Store

Steps to Publish:

23. Create a Google Play Developer Account — pay one-time \$25 registration fee at play.google.com/apps/publish
24. Prepare the App:
 - Ensure app has no crashes or obvious bugs
 - Test on multiple devices and screen sizes
 - Set `targetSdkVersion` to latest stable
25. Generate Signed APK / AAB:
 - In Android Studio: Build → Generate Signed Bundle/APK
 - Create or use existing Keystore file
 - AAB (Android App Bundle) is preferred over APK
26. Create App in Play Console:
 - All apps → Create app → Choose default language, app type
27. Set up Store Listing:
 - App name, Short description, Full description
 - Screenshots (phone, tablet), Feature graphic (1024x500px)
 - App category, Contact email, Privacy policy URL
28. Content Rating — Complete IARC questionnaire
29. Pricing and Distribution — Free or paid, countries
30. Upload AAB — Go to Production track → Create new release → Upload AAB
31. Review and Publish — Submit for review (takes hours to days)
32. App goes Live on Play Store

Q30. App Signing in Android

App Signing is the process of digitally signing an APK or AAB with a private key before publishing. Android requires all apps to be signed.

Why App Signing?

- Proves the app comes from you (authentic author)
- Ensures the app hasn't been tampered with after signing
- Required for Play Store publication
- Only same key can publish updates — losing keystore = cannot update app

Types of Signing:

Debug Signing:

- Android Studio auto-signs debug builds with a debug keystore
- Only for testing — not accepted by Play Store

Release Signing:

- You create a Keystore file with a private key
- Sign release APK/AAB with this keystore

Creating Keystore in Android Studio:

33. Build → Generate Signed Bundle/APK
34. Choose APK or Android App Bundle
35. Create new keystore (or use existing)
36. Fill: Keystore path, Password, Key alias, Key password, Validity (25+ years)
37. Choose release build variant → Finish

Google Play App Signing:

- Google Play now manages the app signing key after upload
- You upload with an upload key; Google re-signs with the app signing key
- Safer — even if upload key is lost, Google can reset it

UNIT 5 — FUNDAMENTALS OF DART PROGRAMMING FOR FLUTTER

Q31. Dart Data Types, Variables, and Constants

What is Dart?

Dart is a strongly-typed, object-oriented programming language developed by Google. It is the primary language for Flutter app development.

Data Types in Dart:

- int — Integer values: `int age = 21;`
- double — Decimal values: `double pi = 3.14;`
- num — Supertype of int and double
- String — Text: `String name = 'Raj';`
- bool — Boolean: `bool isLoggedIn = true;`
- List — Ordered collection: `List<int> nums = [1, 2, 3];`
- Map — Key-value pairs: `Map<String,int> scores = {'Raj': 90};`
- Set — Unordered unique values: `Set<String> tags = {'dart', 'flutter'};`
- dynamic — Any type: `dynamic x = 42; x = 'hello';`
- var — Type inferred: `var city = 'Surat';` (inferred as String)
- Object — Root of all Dart objects

Variables:

```
// Explicit type
int age = 21;
String name = 'Priya';
double salary = 45000.50;

// var - type inferred at compile time
var city = 'Ahmedabad'; // String
var count = 10;          // int

// dynamic - type can change at runtime
dynamic data = 100;
data = 'changed to string'; // valid
```

```
// Nullable - ? allows null
String? nickname = null; // nullable
```

Constants:

final:

- Value assigned once at runtime; cannot be changed after

```
final String username = getUsername(); // can be
set at runtime
username = 'other'; // ERROR - cannot reassign
```

const:

- Compile-time constant — value must be known at compile time
- Deeply immutable — all members are also constant

```
const double PI = 3.14159;
const List<int> primes = [2, 3, 5, 7]; //
compile-time constant list
```

Feature	var	final	const
Type	Inferred	Explicit or inferred	Explicit or inferred
Mutable	Yes	No (once set)	No
When set	Compile time	Runtime (once)	Compile time only
Use case	Normal variable	Single-assignment var	Math constants, configs

Q32 & Q35. Control Flow Statements and Loops in Dart

1. if-else:

```
int marks = 75;
if (marks >= 90) {
  print('Grade: A');
} else if (marks >= 75) {
  print('Grade: B');
} else if (marks >= 60) {
  print('Grade: C');
} else {
```

```
    print('Grade: D');  
}
```

2. switch-case:

```
String day = 'Monday';  
switch(day) {  
    case 'Monday':  
        print('Start of work week');  
        break;  
    case 'Friday':  
        print('Last working day');  
        break;  
    case 'Saturday':  
    case 'Sunday':  
        print('Weekend!');  
        break;  
    default:  
        print('Regular day');  
}
```

3. for loop:

```
for (int i = 1; i <= 5; i++) {  
    print('Count: $i');  
}
```

4. for-in loop (iterate collections):

```
List<String> names = ['Raj', 'Priya', 'Aarav'];  
for (String name in names) {  
    print('Hello $name');  
}
```

5. forEach (functional style):

```
names.forEach((name) => print(name));
```

6. while loop:

```
int n = 1;  
while (n <= 5) {  
    print(n);  
    n++;  
}
```

7. do-while loop:

```
int x = 0;
do {
  print('x = $x');
  x++;
} while (x < 3);
```

8. break and continue:

```
for (int i = 0; i < 10; i++) {
  if (i == 3) continue; // skip 3
  if (i == 7) break;    // stop at 7
  print(i);
}
```

Q33, Q34 & Q36. OOP in Dart — Classes, Objects, Functions, Constructors

Functions in Dart:

A function is a reusable block of code that performs a specific task. Dart functions are first-class objects.

```
// Basic function
int add(int a, int b) {
  return a + b;
}

// Arrow function (single expression)
int multiply(int a, int b) => a * b;

// Named parameters (with {})
void greet({String name = 'Guest', int age = 0}) {
  print('Hello $name, age $age');
}
greet(name: 'Raj', age: 21);

// Optional positional parameters (with [])
void show(String city, [String country = 'India'])
{
  print('$city, $country');
```

```
}
```

Class and Object in Dart:

A class is a blueprint for creating objects. An object is an instance of a class.

```
class Student {
  // Fields (properties)
  String name;
  int marks;

  // Default constructor
  Student(this.name, this.marks);

  // Named constructor
  Student.empty() : name = 'Unknown', marks = 0;

  // Method
  void display() {
    print('Student: $name | Marks: $marks');
  }

  // Getter
  String get grade => marks >= 75 ? 'A' : marks >=
60 ? 'B' : 'C';
}

void main() {
  // Creating object
  Student s1 = Student('Priya', 88);
  s1.display();
  print('Grade: ${s1.grade}');

  Student s2 = Student.empty(); // named
constructor
  s2.display();
}
```

Inheritance in Dart:

```
class Animal {
```

```
String name;  
Animal(this.name);  
void sound() => print('$name makes a sound');  
}  
  
class Dog extends Animal {  
    Dog(String name) : super(name);  
    @override  
    void sound() => print('$name says: Woof!');  
}  
  
void main() {  
    Dog d = Dog('Bruno');  
    d.sound(); // Bruno says: Woof!  
}
```

UNIT 7 — WORKING WITH UI IN FLUTTER

Q43 & Q46. Flutter Widgets — Stateless and Stateful

In Flutter, everything is a Widget. A Widget is the basic building block of a Flutter UI — it describes what the UI should look like given its current configuration and state.

What is a Widget?

- Immutable description of part of the user interface
- Flutter rebuilds widget tree when state changes
- Widgets are nested to form the UI tree

Types of Widgets:

1. Stateless Widget:

- Has no mutable state — UI never changes after build
- Rebuilt only when parent changes its configuration
- Extends StatelessWidget, overrides build()

```
class GreetingWidget extends StatelessWidget {
  final String name;
  const GreetingWidget({required this.name});

  @override
  Widget build(BuildContext context) {
    return Text(
      'Hello, $name!',
      style: TextStyle(fontSize: 24),
    );
  }
}
```

2. Stateful Widget:

- Has mutable state — UI can change dynamically
- Extends StatefulWidget; state is held in State<T> class
- Call setState() to update UI

```
class CounterWidget extends StatefulWidget {
  @override
```

```

    _CounterWidgetState createState() =>
    _CounterWidgetState();
}

class _CounterWidgetState extends
State<CounterWidget> {
    int count = 0;

    @override
    Widget build(BuildContext context) {
        return Column(
            children: [
                Text('Count: $count', style:
TextStyle(fontSize: 24)),
                ElevatedButton(
                    onPressed: () => setState(() =>
count++),
                    child: Text('Increment'),
                ),
            ],
        );
    }
}

```

Stateless vs Stateful Comparison:

Feature	StatelessWidget	StatefulWidget
State	No mutable state	Has mutable state
Rebuilds when	Parent rebuilds it	setState() is called
Performance	Better (simpler)	Slightly heavier
Use case	Static UI (text, icon)	Dynamic UI (counter, form, toggle)
Key method	build()	build() + setState()
Classes needed	1	2 (widget + state)

Q44 & Q48. Layout Design — Container, Row, and Column

Container Widget:

Container is a convenience widget that combines common painting, positioning, and sizing. It's like a box that wraps a child widget.

- width, height — Set dimensions
- color — Background color
- padding — Space inside the container
- margin — Space outside the container
- decoration — Border, border radius, gradient, shadow
- alignment — Align child inside container

```
Container(  
    width: 200,  
    height: 100,  
    margin: EdgeInsets.all(10),  
    padding: EdgeInsets.symmetric(horizontal: 16,  
vertical: 8),  
    decoration: BoxDecoration(  
        color: Colors.blue,  
        borderRadius: BorderRadius.circular(12),  
        boxShadow: [BoxShadow(color: Colors.grey,  
blurRadius: 4)],  
    ),  
    child: Text('Hello', style: TextStyle(color:  
Colors.white)),  
)
```

Row Widget:

Row arranges children horizontally (left to right). Equivalent to horizontal `LinearLayout` in Android.

- `mainAxisAlignment` — How children are spaced horizontally
- `crossAxisAlignment` — How children are aligned vertically

```
Row(  
    mainAxisAlignment:  
MainAxisAlignment.spaceEvenly,  
    crossAxisAlignment: CrossAxisAlignment.center,  
    children: [  

```

```

        Icon(Icons.star, color: Colors.yellow),
        Text('Flutter'),
        ElevatedButton(onPressed: () {}, child:
Text('Click')),
    ],
)

```

Column Widget:

Column arranges children vertically (top to bottom). Equivalent to vertical `LinearLayout` in Android.

```

Column(
  mainAxisAlignment: MainAxisAlignment.center,
  crossAxisAlignment: CrossAxisAlignment.stretch,
  children: [
    Text('Title', style: TextStyle(fontSize: 24,
fontWeight: FontWeight.bold)),
    SizedBox(height: 16),
    TextField(decoration:
InputDecoration(labelText: 'Enter name')),
    SizedBox(height: 16),
    ElevatedButton(onPressed: () {}, child:
Text('Submit')),
  ],
)

```

MainAxisAlignment Values:

Value	Effect
start	All children at start
end	All children at end
center	All children at center
spaceBetween	Equal space between children, no edge space
spaceAround	Equal space around each child
spaceEvenly	Equal space everywhere including edges

Q45. Themes and Styling in Flutter

Flutter's Theme system allows defining a consistent visual style (colors, fonts, shapes) across the entire app from one place.

Why Themes?

- Consistency — One change affects the whole app
- Reusability — No need to repeat styles in every widget
- Dark mode support — Easy switching between light and dark themes

Setting Theme in MaterialApp:

```
MaterialApp(  
  title: 'My App',  
  theme: ThemeData(  
    primarySwatch: Colors.blue,  
    primaryColor: Color(0xFF3F51B5),  
    colorScheme: ColorScheme.fromSeed(seedColor:  
Colors.blue),  
    textTheme: TextTheme(  
      displayLarge: TextStyle(fontSize: 32,  
fontWeight: FontWeight.bold),  
      bodyLarge: TextStyle(fontSize: 16, color:  
Colors.black87),  
    ),  
    elevatedButtonTheme: ElevatedButtonThemeData(  
      style: ElevatedButton.styleFrom(  
        backgroundColor: Colors.blue,  
        shape: RoundedRectangleBorder(  
          borderRadius: BorderRadius.circular(8)),  
      ),  
    ),  
    darkTheme: ThemeData.dark(),  
    themeMode: ThemeMode.system, // follows device  
setting  
    home: HomeScreen(),  
  )  
)
```

Accessing Theme in Widgets:

```
Text(  
  // Accessing theme here  
)
```

```
'Hello',  
  style: Theme.of(context).textTheme.displayLarge,  
)  
Color primary = Theme.of(context).primaryColor;
```

Custom TextStyle:

```
Text(  
  'Flutter is Amazing',  
  style: TextStyle(  
    fontSize: 20,  
    fontWeight: FontWeight.bold,  
    color: Colors.indigo,  
    letterSpacing: 1.5,  
    fontFamily: 'Roboto',  
  ),  
)
```

Q47. SnackBar and Dialogs in Flutter

SnackBar:

A SnackBar is a lightweight message shown briefly at the bottom of the screen. Used for quick feedback (like 'Item deleted', 'Saved').

- Appears at bottom of screen
- Auto-dismisses after a few seconds
- Can have an optional action button (Undo)

```
// Show a SnackBar  
ScaffoldMessenger.of(context).showSnackBar(  
  SnackBar(  
    content: Text('Item deleted!'),  
    duration: Duration(seconds: 3),  
    backgroundColor: Colors.red,  
    action: SnackBarAction(  
      label: 'Undo',  
      textColor: Colors.white,  
      onPressed: () {  
        // Undo the delete  
      },  
    ),  
  ),  
)
```

```

    ),
  ),
)

```

AlertDialog:

AlertDialog is a popup dialog used for confirmation, warnings, or information.

```

showDialog(
  context: context,
  builder: (ctx) => AlertDialog(
    title: Text('Confirm Delete'),
    content: Text('Are you sure you want to delete
this item?'),
    actions: [
      TextButton(
        onPressed: () => Navigator.of(ctx).pop(),
        child: Text('Cancel'),
      ),
      ElevatedButton(
        onPressed: () {
          deleteItem();
          Navigator.of(ctx).pop();
        },
        child: Text('Delete'),
        style:
ElevatedButton.styleFrom(backgroundColor:
Colors.red),
      ),
    ],
  ),
)

```

SimpleDialog:

SimpleDialog shows a list of options for the user to choose from.

```

showDialog(
  context: context,
  builder: (ctx) => SimpleDialog(
    title: Text('Choose Color'),
    children: [

```

```

        SimpleDialogOption(
            onPressed: () { Navigator.pop(ctx); },
            child: Text('Red'),
        ),
        SimpleDialogOption(
            onPressed: () { Navigator.pop(ctx); },
            child: Text('Blue'),
        ),
    ],
),
)

```

BottomSheet:

```

showModalBottomSheet(
    context: context,
    builder: (ctx) => Container(
        height: 200,
        child: Column(children: [
            ListTile(leading: Icon(Icons.share), title:
Text('Share')),
            ListTile(leading: Icon(Icons.delete), title:
Text('Delete')),
        ]),
    ),
)

```

Dialog Type	Purpose	Dismissal
SnackBar	Brief feedback message	Auto (or swipe)
AlertDialog	Confirmation / Warning	Button or tap outside
SimpleDialog	Selection from options	Option tap or back
BottomSheet	Action sheet from bottom	Swipe or tap outside

— END OF MAD EXAM GUIDE — Best of Luck!